EE2213 Introduction to Artificial Intelligence

Lecture 6

Dr. Shaojing Fan
fanshaojing@nus.edu.sg

OVERVIEW OF COURSE CONTENTS

- **Introduction (Shaojing)**
  - What is AI
  - Applications of AI
  - AI agent
- **Search (Shaojing)**
  - Uninformed search algorithms: breadth-first, depth-first, uniform-cost(Dijkstra's algorithm)
  - Informed search algorithms: greedy best-first, A*
  - Applications
- **Optimization (Shaojing)**
  - Linear programming
  - **Convex problems**
  - **Applications**
- **Machine learning (Wang Si)**
  - Supervised and unsupervised learning: regression, classification, clustering
  - Neural networks and deep learning
  - Applications
- **Knowledge representation (Wang Si)**
  - Knowledge Representation and Reasoning
  - Propositional Logic
  - Applications
- **Ethical considerations (Shaojing)**
  - Bias in AI
  - Privacy concerns
  - Societal impact

Path optimization

Mathematical optimization

Recap: Linear programs (LP)

An optimization problem that aims to **maximize or minimize a linear objective function**, subject to **linear equality and/or inequality constraints**.

$$\begin{aligned} &\text{Maximize} && \mathbf{c}^T\mathbf{x} \\ &\text{Subject to} && \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & && \mathbf{x} \geq 0 \end{aligned}$$

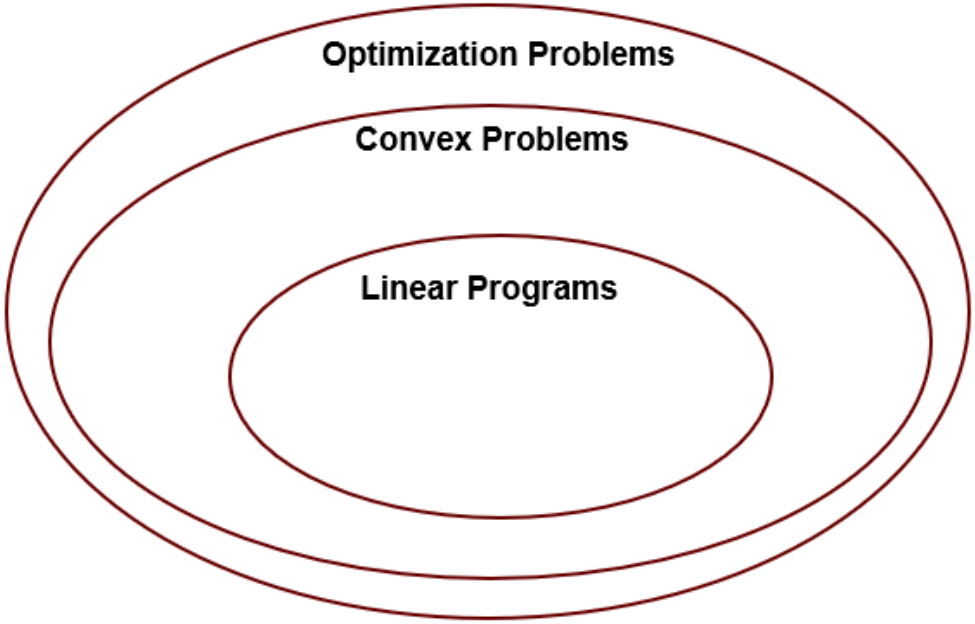
Solving LPs:

Graphical method: applicable when there are only two variables.

Simplex algorithm: efficient for large-scale LPs

Fundamental theorem in LP: Optimal solutions occur on the boundary of the feasible region — typically at its vertices (or along an edge between vertices).

Linear programs vs convex problems



Linear programming is probably the most widely studied and broadly useful method for optimization. It is a special case of the more general problem of **convex optimization**, which allows the constraint region to be any convex region and the objective to be any function that is convex within the constraint region. Under certain conditions, convex optimization problems are also polynomially solvable and may be feasible in practice with thousands of variables. Several important problems in machine learning and control theory can be formulated as convex optimization problems (see Chapter 21).

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P140

Agenda

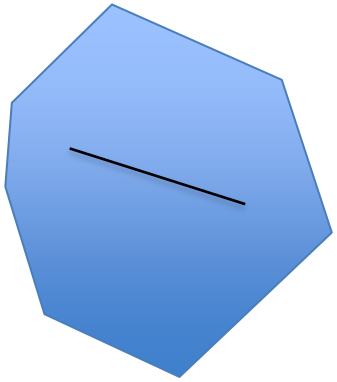
- Background: Convex sets and convex functions
- Convex optimization problems: Real-world examples
- Solving convex problems: Gradient descent

Convex Optimization
Stephen Boyd and Lieven Vandenberghe

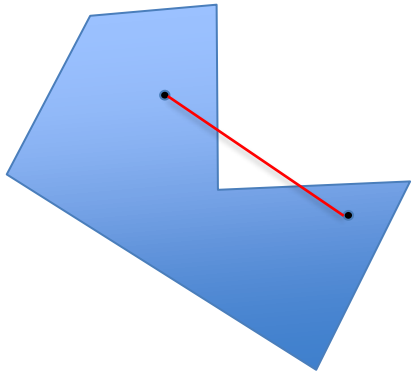
Cambridge University Press

Reference (modern bible of convex optimization): Stephen Boyd & Lieven Vandenberghe, Convex Optimization, https://web.stanford.edu/~boyd/cvxbook/

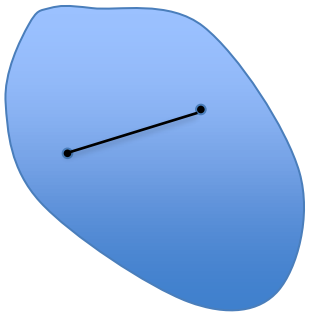
Convex set: Definition



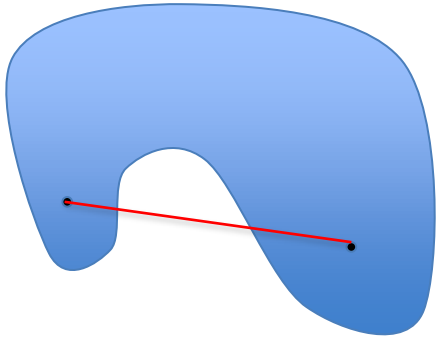
Convex



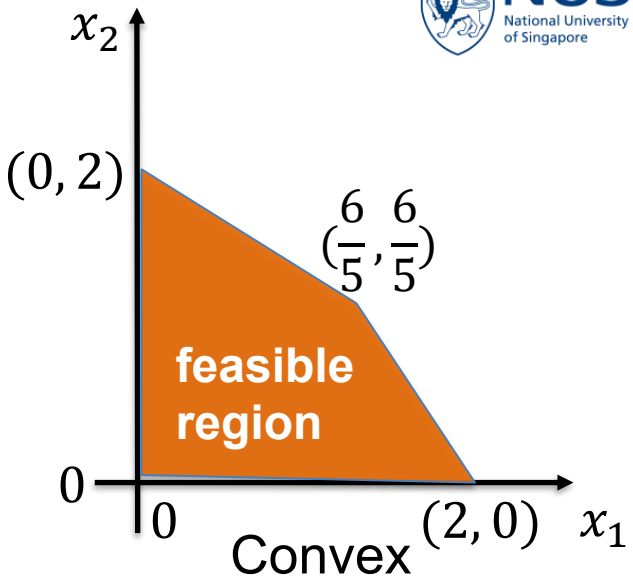
Non-convex



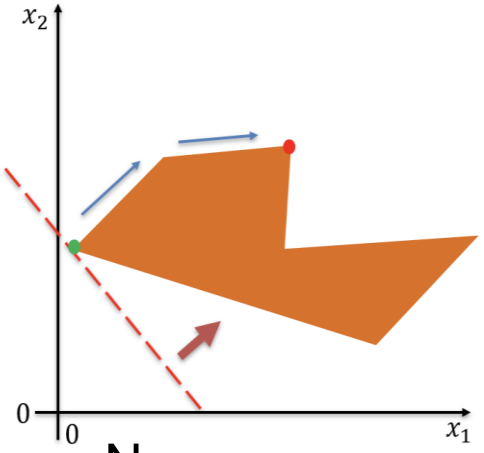
Convex



Non-convex



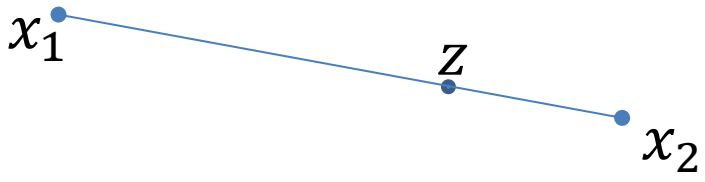
Convex

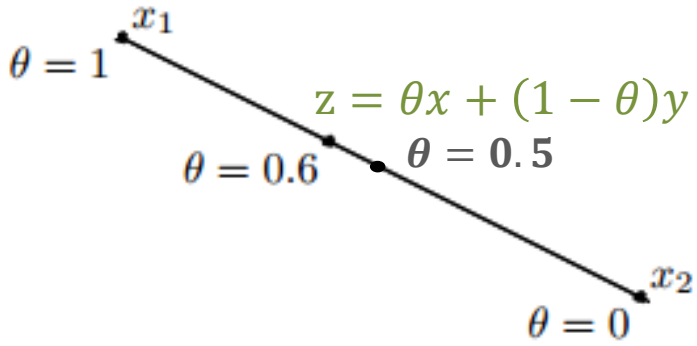


Non-convex

Convex combination: Definition

- A point between two points
- Given $x_1, x_2 \in \mathbb{R}^n$, a *convex combination* of them is any point of the form $z = \theta x_1 + (1-\theta)x_2$ where $\theta \in [0,1]$
- When $\theta \in (0,1)$, z is called a strict convex combination of x_1, x_2

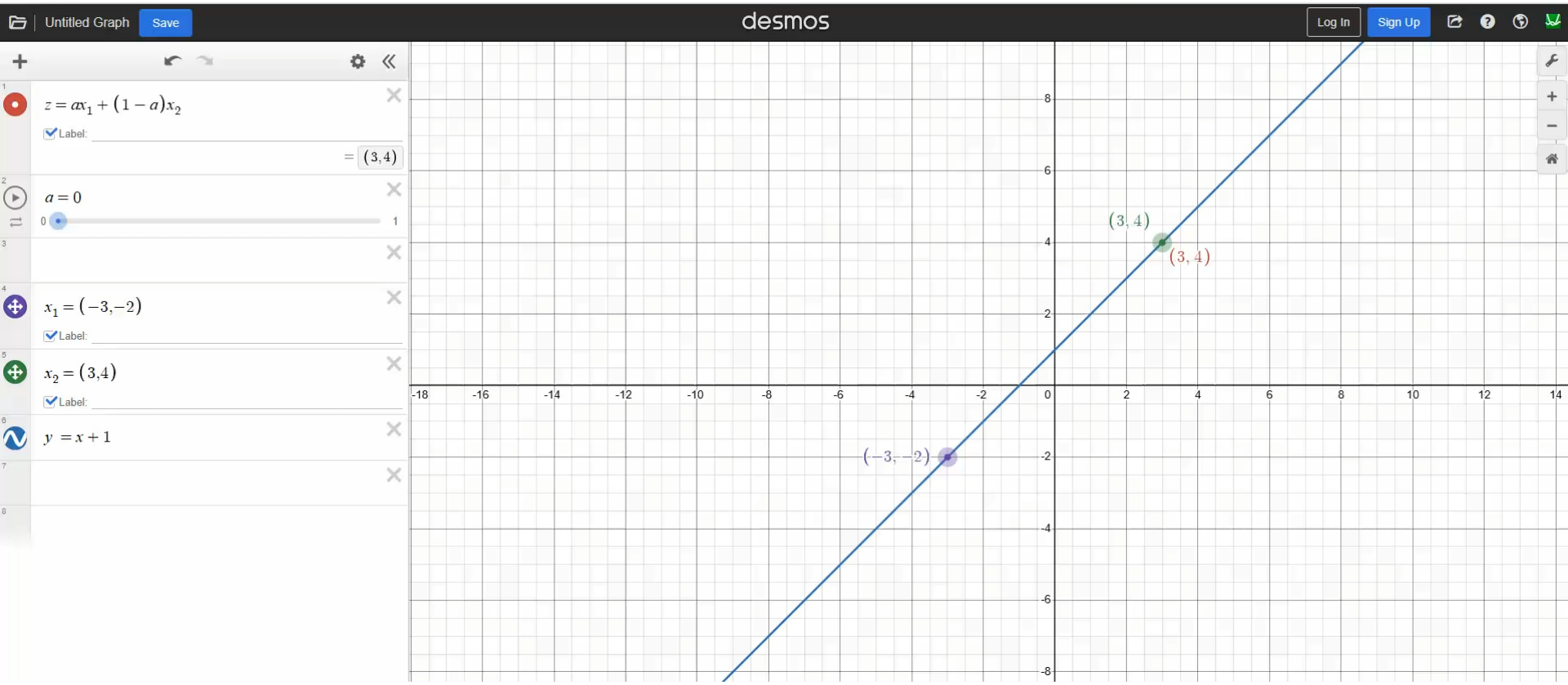




A convex combination is just a fancy way of saying "a point between two points."

Convex combination: Definition

Given $x_1, x_2 \in \mathbb{R}^n$, a *convex combination* of them is any point of the form $z = \theta x_1 + (1 - \theta) x_2$ where $\theta \in [0,1]$

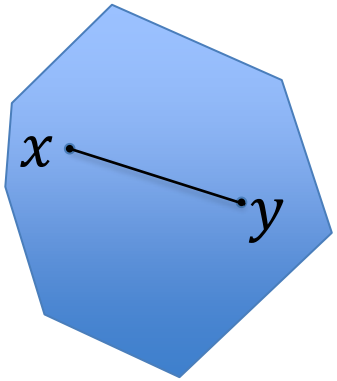


Animation: https://www.desmos.com/calculator

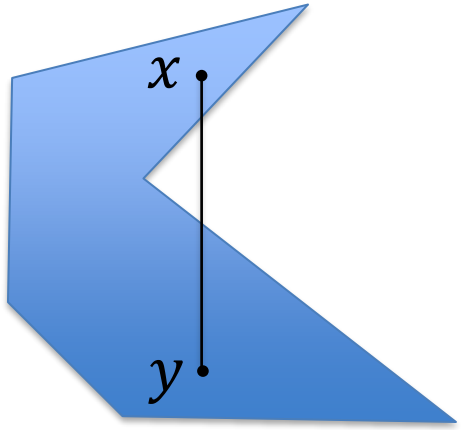
Convex set: Definition

A convex set is a set $C \subseteq R^n$ such that for all $x_1, x_2 \in C$ and $\theta \in [0,1]$ we have :

$$\theta x_1 + (1-\theta)x_2 \in C$$



Convex set



Non-convex set

Quick question 1

Which of the following sets are convex?

A: $C = \mathbb{R}^n$

B: $C = \emptyset$

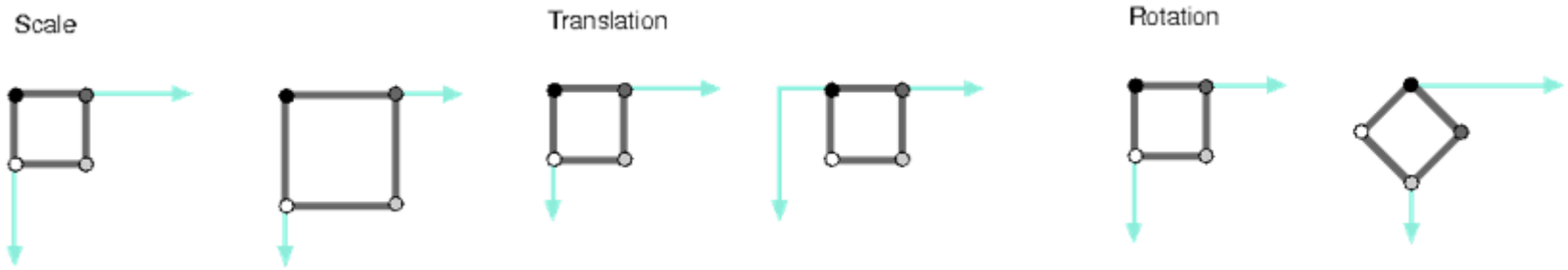
C: $C = \{x_0\},\ x_0 \in \mathbb{R}^n$

D: $C = C_1 \cap C_2$, where C_1 and C_2 are convex sets

E: $C = C_1 \cup C_2$, where C_1 and C_2 are convex sets

Convex set: Properties

- The empty set $\emptyset$ and $\mathbb{R}^n$ are both convex.
- Intersections of convex sets are convex.
- Convexity is **preserved** under three common operations:
  - **Scaling:** Multiplying all points by a positive scalar
  - **Translation:** Shifting to a new position (adding a constant vector to all points)
  - **Rotation:** Rotating them around an axis



Convex function: Definition

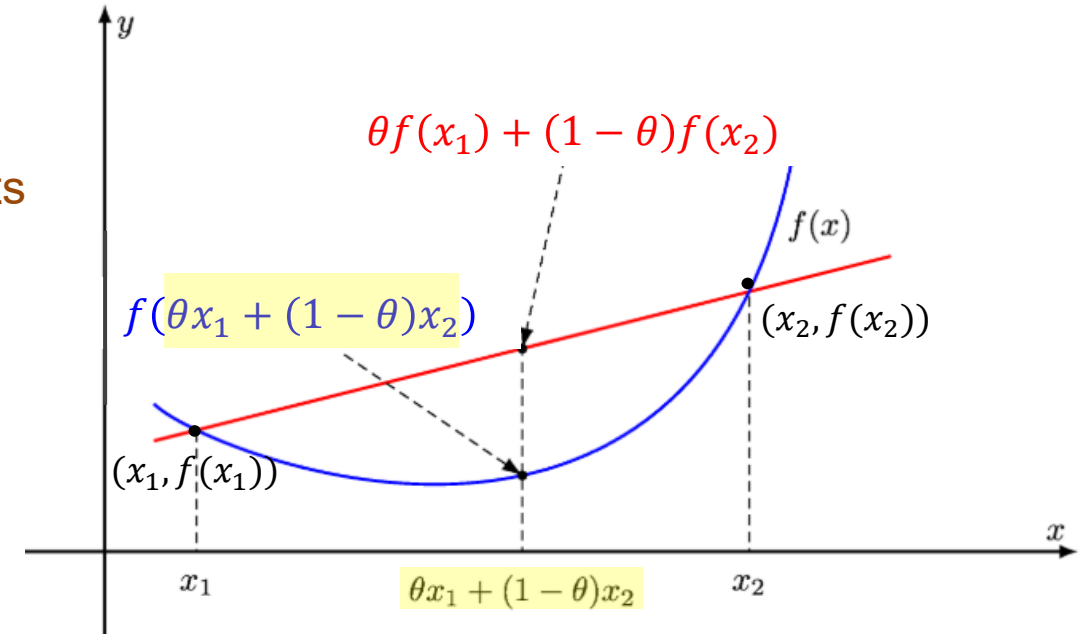
Let C be a convex set. A function $f: C \to \mathbb{R}$ is convex in C if $\forall x_1, x_2 \in C, \forall \theta \in [0,1]$,

$$f(\theta x_1 + (1-\theta)x_2) \leq \theta f(x_1) + (1-\theta) f(x_2)$$

If $C = \mathbb{R}^n$, we simply say f is a convex function.

Any chord between two points on the function always lies above the function.

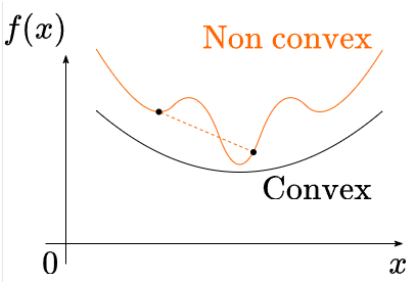
Value in the middle point is lower than the average of the two endpoint values.



Convex function: Definition (cont.)

Let C be a convex set. A function $f: C \to \mathbb{R}$ is convex in C if $\forall x_1, x_2 \in C, \forall \theta \in [0,1]$,

$$f(\theta x_1 + (1-\theta)x_2) \leq \theta f(x_1) + (1-\theta) f(x_2)$$



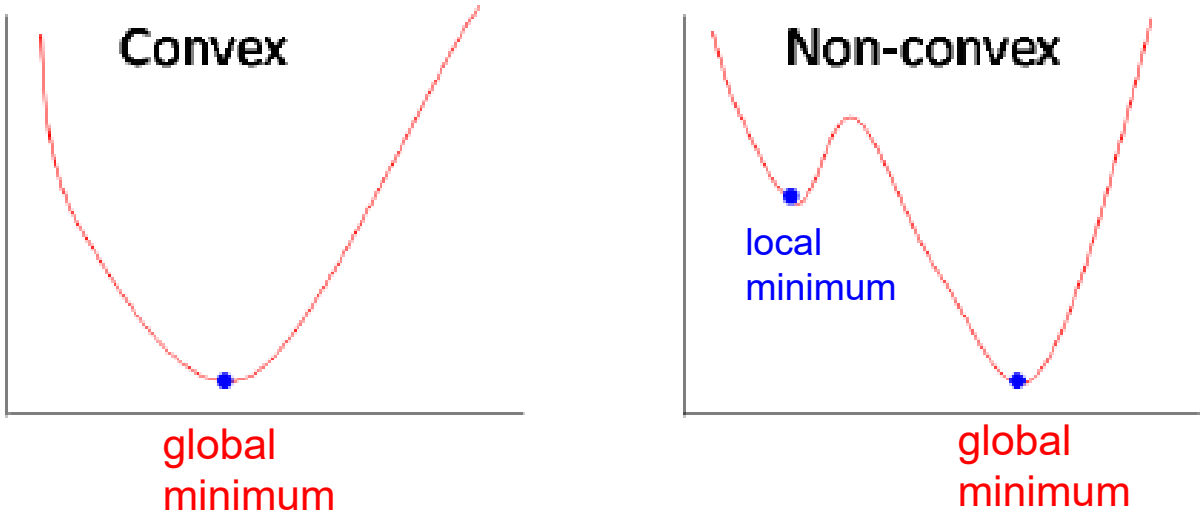
If $C = \mathbb{R}^n$, we simply say f is a convex function.

Any chord between two points on the function always lies above the function.

Convex function: Properties

- Sum of convex functions is convex
  - $af(x) + bg(x)$ is convex for convex functions f, g and $a, b > 0$.
- Convexity is preserved under affine transformation
  - If $f(\mathbf{x}) = g(\boldsymbol{A}\mathbf{x} + \mathbf{b})$, g is convex, then $f(\mathbf{x})$ is convex.

  where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$
- Any local minimum is a global minimum



Recap: Affine transformation

Definition:
An affine transformation is a fundamental concept in linear algebra and geometry. It's a type of transformation that preserves points, straight lines, and planes. Crucially, it also preserves parallelism (lines that are parallel before the transformation remain parallel after).

An affine transformation is a combination of a **linear transformation** (like rotation, scaling, reflection) and a **translation**(shifting).

$$\mathbf{y} = \boldsymbol{A}\mathbf{x} + \mathbf{b}$$

where $\boldsymbol{A}$: matrix (rotation, scaling, etc.)
$\mathbf{b}$: vector (shift)

Why important?

- Convexity is preserved under affine transformations.
- That means if a set or function is convex, and you apply *scaling, rotation, or translation*, it **stays convex**.

Recap: Affine transformation

Example of affine transformation: Rotation the **input space**

Take the convex function: $f(x_1, x_2) = {x_1}^2 + 2{x_2}^2$
Let's rotate the ***input*** by 45 degrees. The rotation matrix is

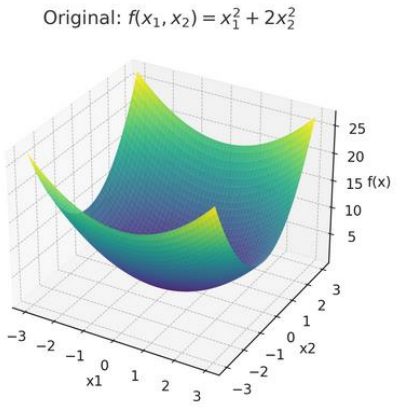
$$\boldsymbol{R} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

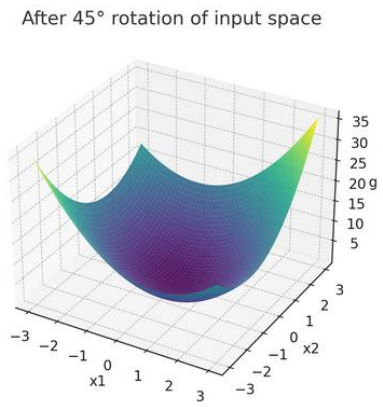
Rotate the input: $\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \boldsymbol{R}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix} x_1 - x_2 \\ x_1 + x_2 \end{bmatrix}$, then $u_1 = \frac{x_1 - x_2}{\sqrt{2}}$, $u_2 = \frac{x_1 + x_2}{\sqrt{2}}$

Define the rotated function $g(\mathbf{x}) = f(\boldsymbol{R}\mathbf{x}) = f(\mathbf{u})$, $g(\mathbf{x}) = {u_1}^2 + 2{u_2}^2$

Substitute u_1, u_2 and expand

$$g(\mathbf{x}) = (\frac{x_1 - x_2}{\sqrt{2}})^2 + 2(\frac{x_1 + x_2}{\sqrt{2}})^2 = \frac{3}{2}{x_1}^2 + x_1 x_2 + \frac{3}{2}{x_2}^2$$





Tools: https://www.desmos.com/3d

Quick question 2

Which of the following functions are convex?

A: $f(x) = x^2 + 2$

B: $f(x) = 3x + 4$

C: $f(x) = \sin(x)$

D: $f(x) = 3$

E: $f(x) = -3x^2$

Examples of convex functions

Linear/affine functions:

$$f(\mathbf{x}) = \mathbf{a}^T\mathbf{x} + b$$

where $\mathbf{a},\ \mathbf{x} \in \mathcal{R}^n$, $b \in \mathcal{R}$

Quadratic functions:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{c}^T\mathbf{x} + d$$

where $\mathbf{a},\ \mathbf{c},\ \mathbf{x} \in \mathcal{R}^n$, $d \in \mathcal{R}$, $\mathbf{Q} \in \mathcal{R}^{n\times n}$ is a symmetric matrix

Example: $f(x) = 2x^2 + 7x - 3$

Agenda

- Background: Convex sets and convex functions
- Convex optimization problems: Real-world examples
- Solving convex problems: Gradient descent

Convex optimization problem: Example 1

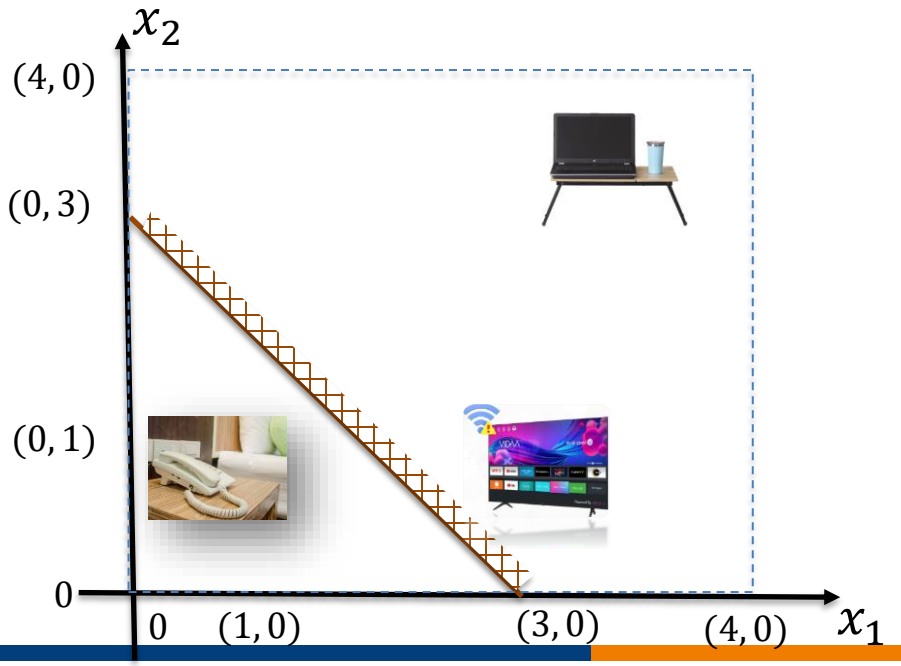
Problem (Wi-Fi placement to serve many devices)

You're placing one Wi-Fi router in a 4 × 4 m room. There are m devices at known locations $u_1, \ldots u_m \in \mathcal{R}^2$ (phones, laptops, smart TV, etc). You want the router to give good average performance, so you minimize the sum of squared distances $(dist_{Euclidean})$ from the router position, $x = (x_1, x_2)$ to the devices. The router must be placed inside the living area: the half-plane behind the divider $x_1 + x_2 \geq 3$ and inside the room $0 \leq x_1, x_2 \leq 4$

Formulation:

$$\min_{x \in \mathcal{R}^2} f(x) = \sum_{i=1}^{m} \|x - u_i\|_2^2$$

$$\text{s. t. } x_1 + x_2 \geq 3,$$
$$0 \leq x_1 \leq 4$$
$$0 \leq x_2 \leq 4$$



$\|x - u_i\|_2 = \sqrt{(x_1 - u_{i1})^2 + (x_2 - u_{i2})^2}$ (the Euclidean distance between the router and device i)

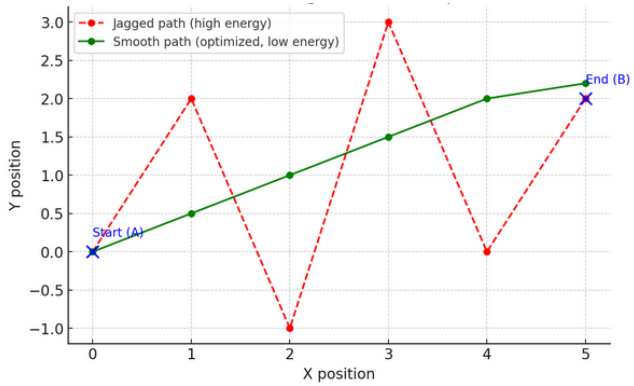
Convex optimization problem: Example 2

Problem (Minimizing energy cost in robot path planning)
Example: A delivery robot wants to move from point A to point B smoothly, using *as little energy as possible*. *Sharp turns cost more energy*.

Minimizing the sum of squared differences in positions, which *penalizes rapid movement changes*:

$$\min_{\mathbf{x}} \sum_{i=2}^{n-1} \|\mathbf{x}_{i+1} - 2\mathbf{x}_i + \mathbf{x}_{i-1}\|^2$$

$\mathbf{x}_i$: the robot's position at step i



$\|x - y\|_1 = the\ Manhatton\ norm\ (a.k.a.\ L_1\ norm)$
Second difference: $(\mathbf{x}_{i+1} - \mathbf{x}_i) - (\mathbf{x}_i - \mathbf{x}_{i-1}) = \mathbf{x}_{i+1} - 2\mathbf{x}_i + \mathbf{x}_{i-1}$

Convex optimization problem: Example 3 (machine learning context)

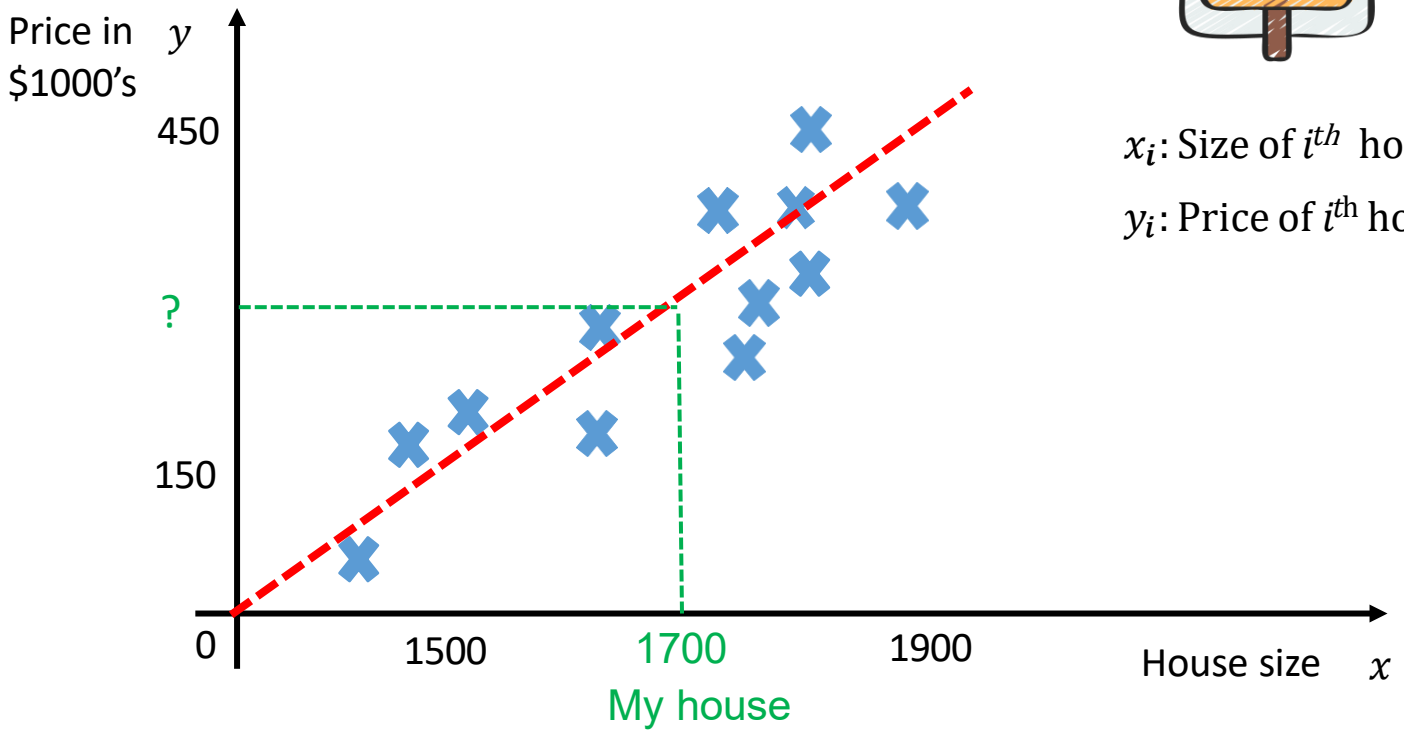
Example: I want to sell my 1,700 ft² house—how much should I ask for?

Historical records:

House size (feet^2)	Sold price ($1000)
1227	128
1360	145
1600	260
1780	306
1900	450
2107	460
…..	…..

Convex optimization problem: Example 3 (machine learning context)

Example: I want to sell my 1,700 ft² house—how much should I ask for?



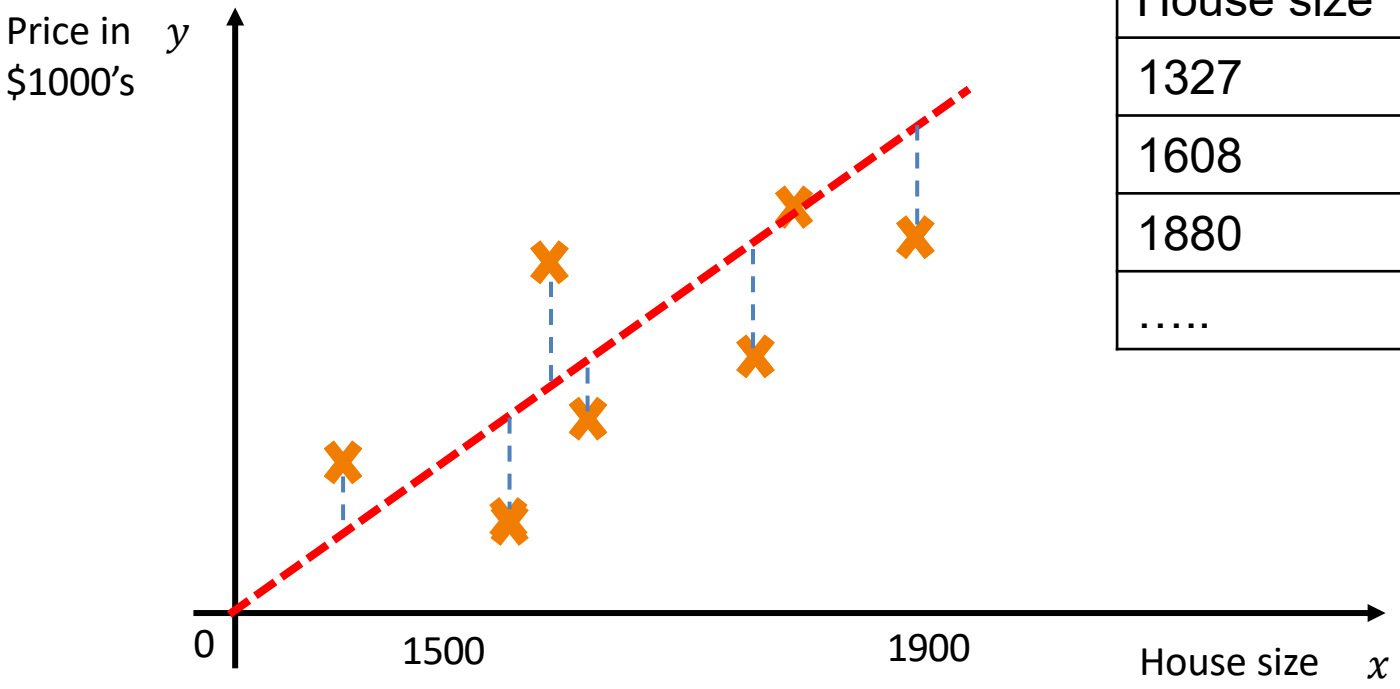
x_i: Size of i^{th} house

y_i: Price of i^{th} house

Convex optimization problem: Example 3 (machine learning context)

How accurate is my prediction?

Historical records (used as validation data):

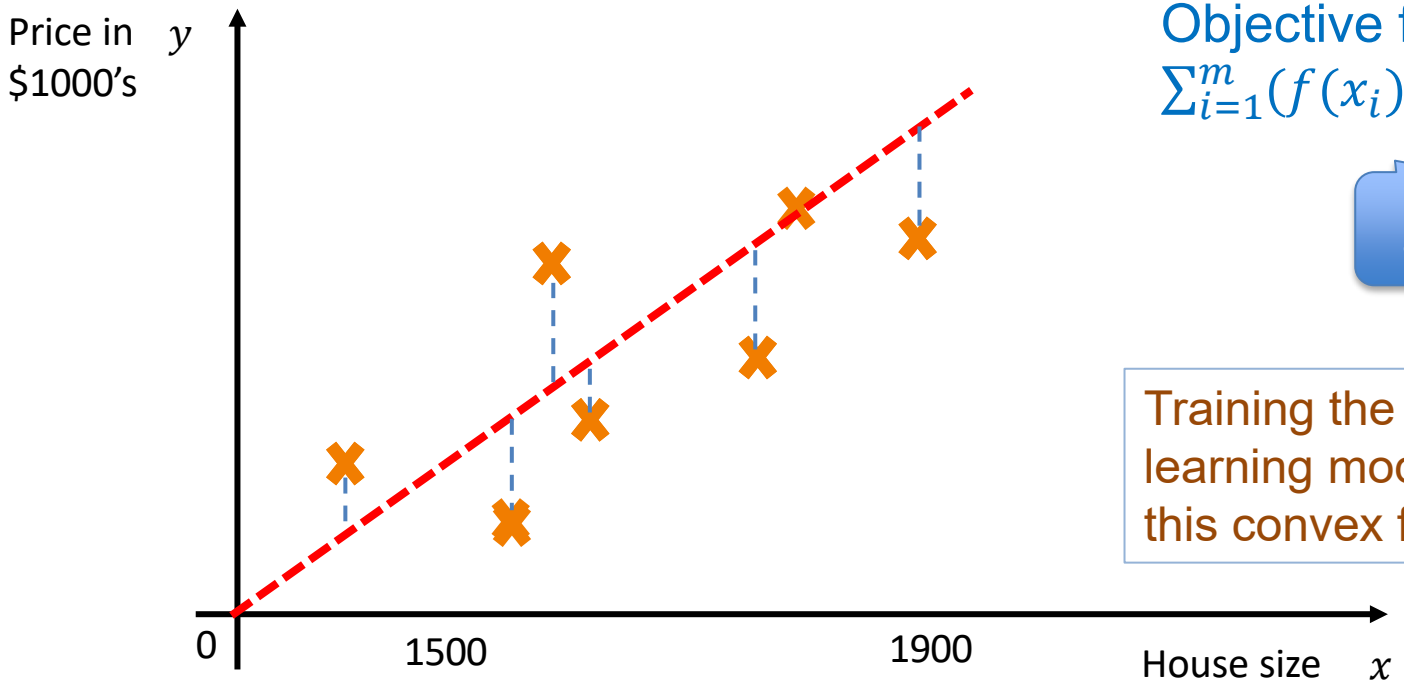


House size	Sold price
1327	148
1608	265
1880	324
…..	…..

Convex optimization problem: Example 3 (machine learning context)

Given m data points (x_1, y_1), (x_2, y_2), (x_m, y_m), linear regression finds a function $f(x)$ (a mapping from x to y) by minimizing the total squared errors $\min_f \sum_{i=1}^{m} (f(x_i) - y_i)^2$

That is, it finds the best-fitting line by minimizing the differences between predicted and actual values.



Objective function:
$\sum_{i=1}^{m} (f(x_i) - y_i)^2$

Convex function

Training the machine learning model = minimizing this convex function

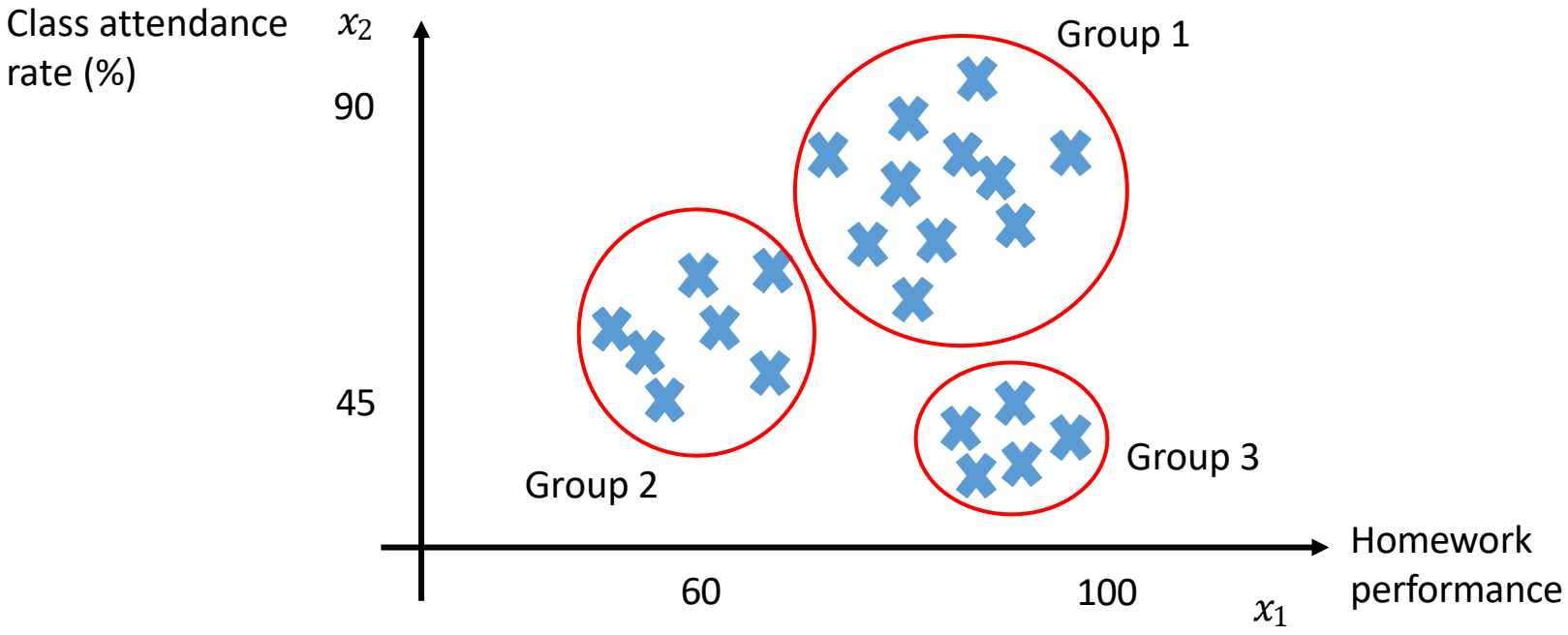
Convex optimization problem: Example 4 (machine learning context)

Example: How many types of students are there in EE2213?

$$x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}$$

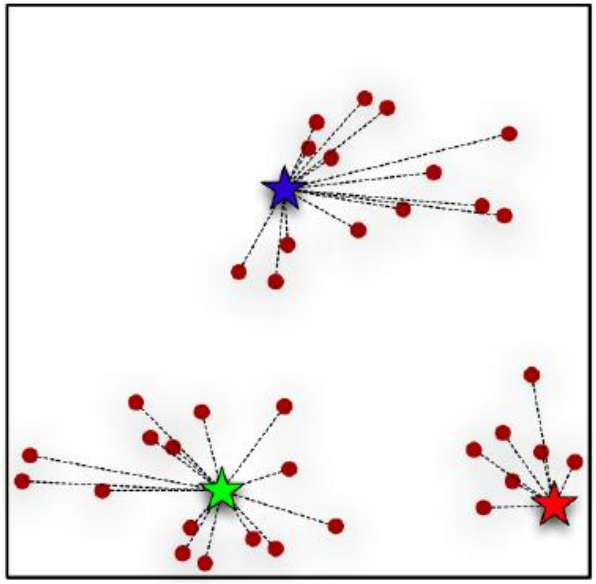
x_{i1}: Homework performance of i^{th} student in EE2213

x_{i2}: Attendance rate of i^{th} student in EE2213



Convex optimization problem: Example 4 (machine learning context)

How good is the clustering result?



We assign each data point x_i to a cluster center c_k , and minimize the total **within-cluster distance**, measured by the **sum of squared distances** between each point and its assigned center:

$$\min_{c_k} \sum_{i=1}^{m} \sum_{k=1}^{K} w_{ik} \|x_i - c_k\|^2$$

Convex function

Training the machine learning model = minimizing this convex function

$w_{ik} = 1$ if point x_i is assigned to cluster k; otherwise 0

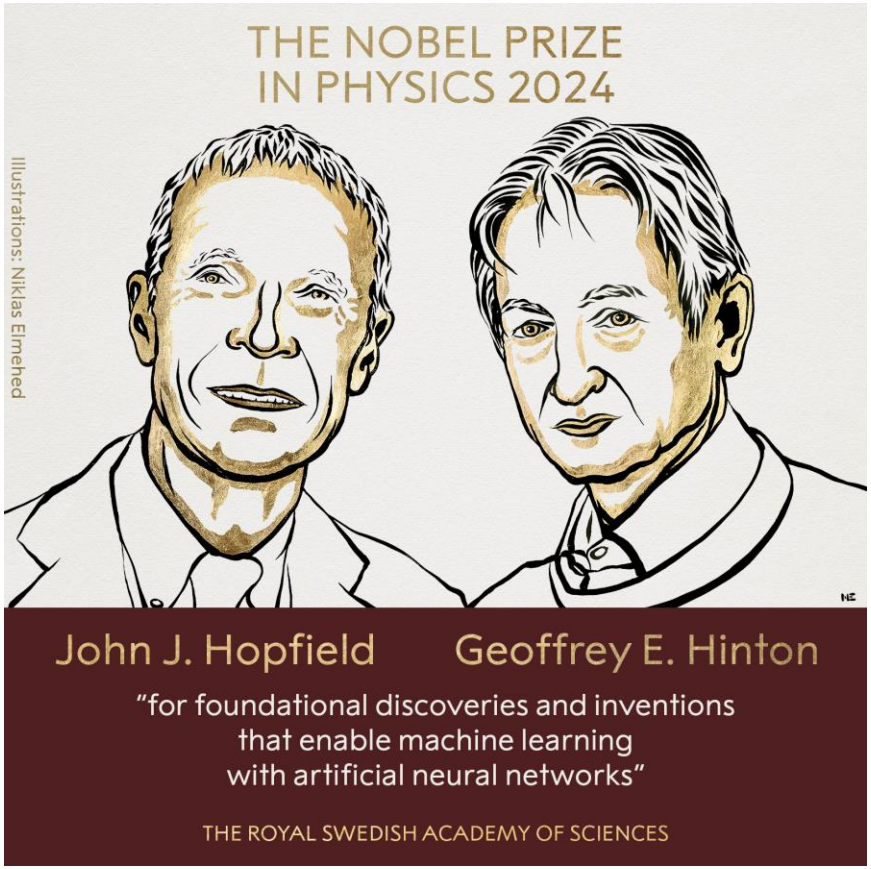
Agenda

- Background: Convex sets and convex functions
- Convex optimization problems: Real-world examples
- Solving convex problems: Gradient descent

Gradient descent: The backbone of neural networks. Empowered for deep learning by Hinton's Nobel Prize

Gradient descent is fundamental to modern AI and neural networks.

2024 Nobel laureate Geoffrey Hinton's work (e.g., backpropagation) enabled gradient descent's power in AI.



Convex optimization: Gradient descent

- Continuous & differentiable setting
  - -- Iteratively update $\mathbf{x}$ to improve the objective
  - -- Use gradient-based method (e.g., gradient descent)

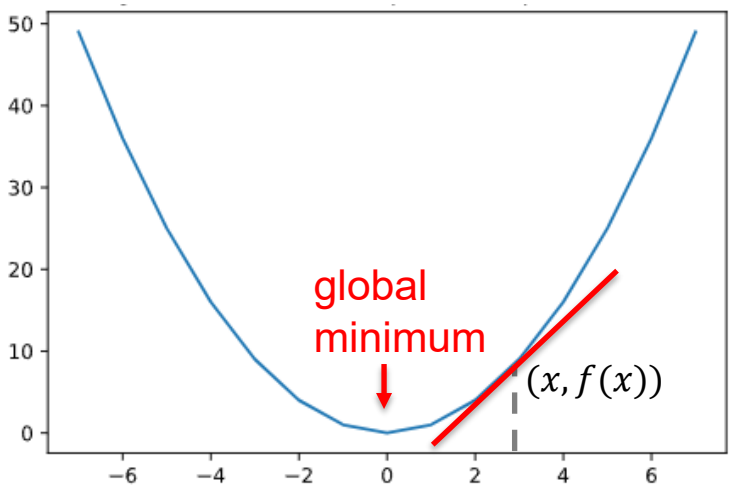
- Discrete setting *(not covered in this course)*
  - -- Local search or integer programming
  - -- Requires different techniques, often problem-specific

Motivation for gradient descent

Goal: Minimize a **convex** objective function $f(\mathbf{x})$

-- $\mathbf{x} \in \mathcal{R}^n$ is a vector of continuous variables

-- $f(\mathbf{x})$ is differentiable



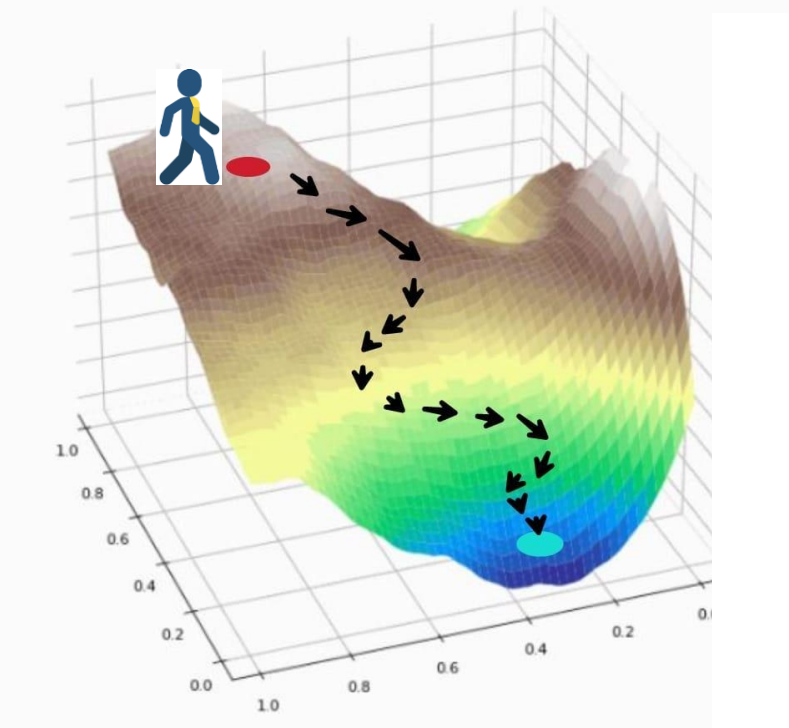
Why use gradient?

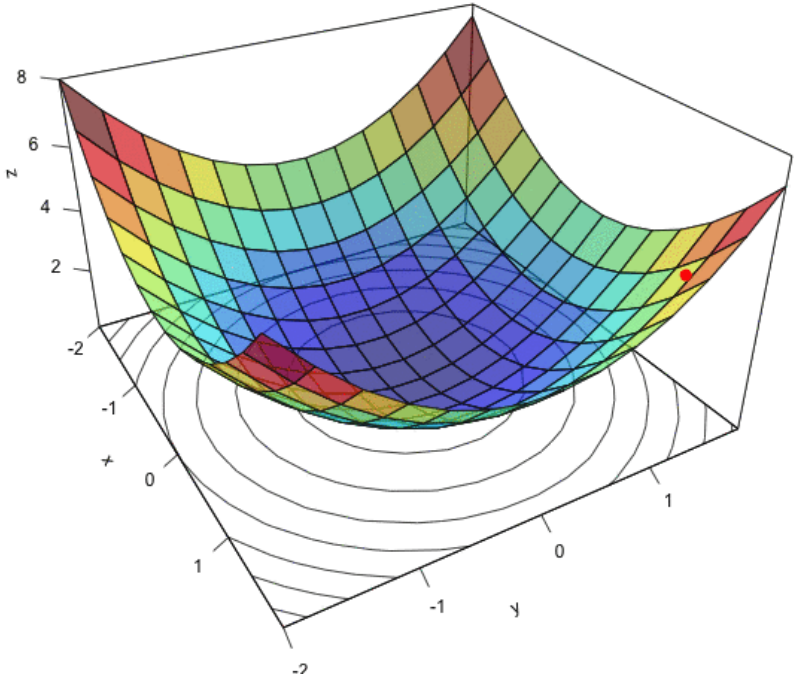
--The **gradient** $\nabla f(\mathbf{x})$ points in the direction of **steepest increase**

--To minimize, we move in the **opposite direction**

Motivation for gradient descent

Gradient descent solves convex optimization by iteratively moving toward the global minimum along the direction of steepest decrease.



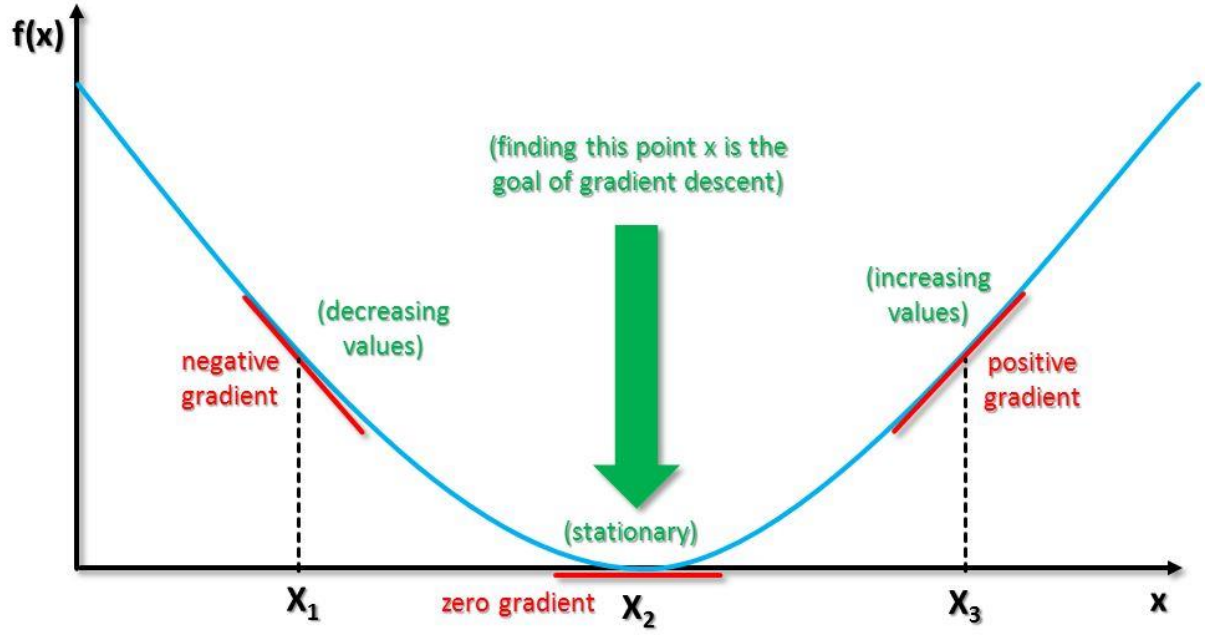


Animation sources: https://www.analyticsvidhya.com/blog/2021/06/the-challenge-of-vanishing-exploding-gradients-in-deep-neural-networks/

Recap: Derivation and gradient

The **derivative** $f'(\mathbf{x})$ of a function $f(\mathbf{x})$ tells how fast the function increases or decreases.

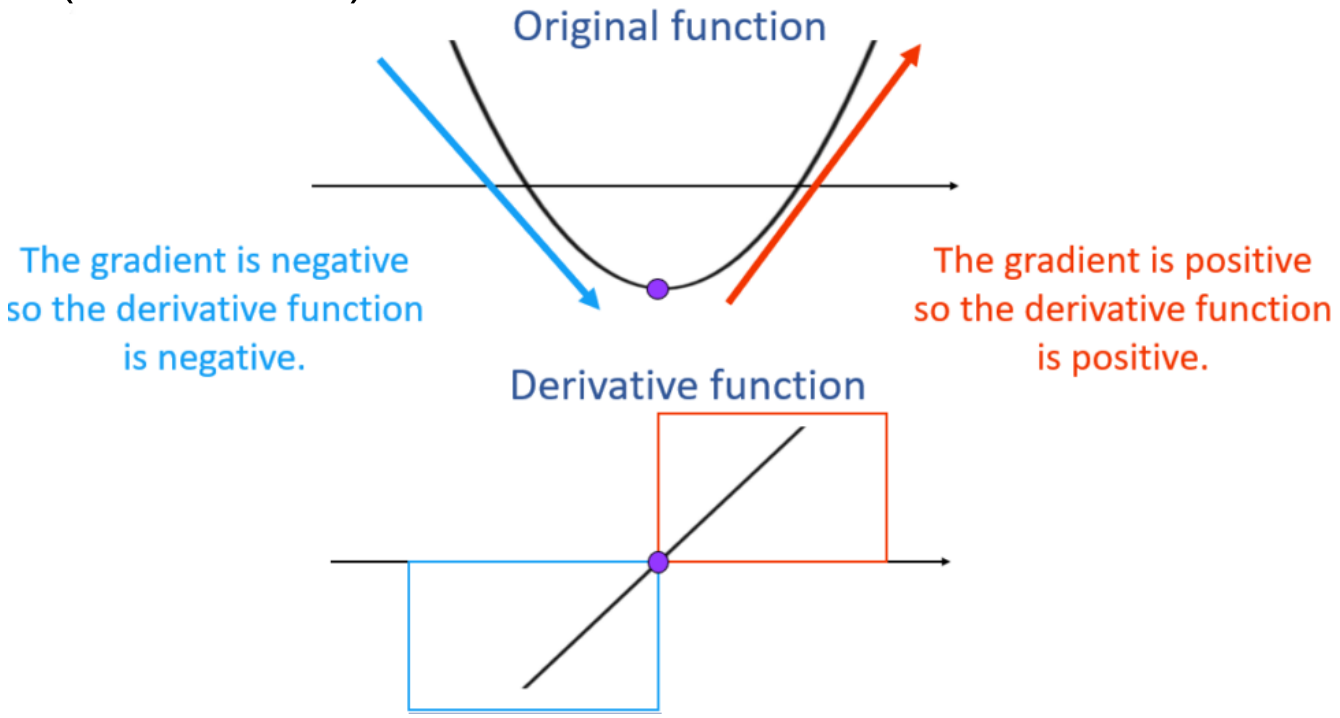
- If $f'(\mathbf{x})$ is constant, the function grows at a *steady* rate.
- If $f'(\mathbf{x}) > 0$, the function is *increasing* at $\mathbf{x}$.
- If $f'(\mathbf{x}) < 0$, the function is *decreasing* at $\mathbf{x}$.
- If $f'(\mathbf{x}) = 0$, the slope is *flat* — possibly a maximum or minimum.



Reference: http://www.big-data.tips/gradient-descent

Recap: Derivation and gradient

- The **derivative** $f'(\mathbf{x})$ of a function $f(\mathbf{x})$ tells how fast the function increases or decreases.
- The process of finding a derivative is called **differentiation.**
- The **gradient** is a generalization of the derivative for functions with **multiple inputs** (*i.e.*, vectors).



Reference: https://mathsathome.com/sketching-the-derivative/

Recap: Derivation and gradient

- The **gradient** of a function is a vector of **partial derivatives**.

If $f(\mathbf{x})$ is a scalar function (*i.e.*, $f: \mathbb{R}^n \rightarrow \mathbb{R}$) of n variables, and $\mathbf{x}$ is a $n \times 1$ vector, then differentiating $f(\mathbf{x})$ with respect to $\mathbf{x}$ yields a $n \times 1$ vector:

$$\frac{df(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

This is called the **gradient** of $f(\mathbf{x})$ and is often denoted by $\nabla_x f$

➢ Example:

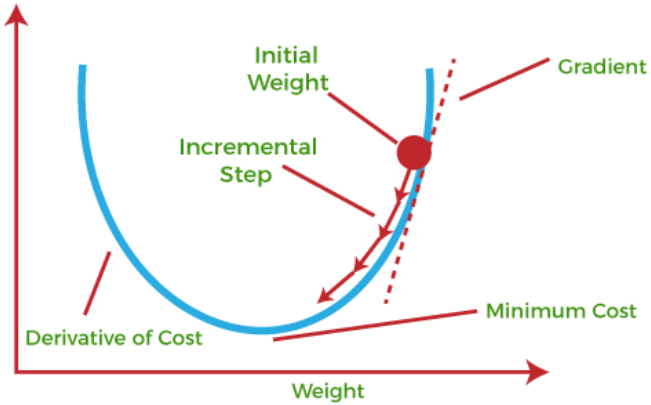
Let $f(\mathbf{x}) = {x_1}^2 + 3x_2$ Then, $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, $\nabla_{\mathbf{x}} f = \begin{bmatrix} 2x_1 \\ 3 \end{bmatrix}$

Reference: https://mathsathome.com/sketching-the-derivative/

Gradient descent algorithm

Suppose we want to minimize $f(\mathbf{x})$ with respect to $\mathbf{x} = [x_1, x_2, \cdots, x_n]^T$

Gradient $\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$



$\nabla_{\mathbf{x}} f(\mathbf{x})$ is a vector and function of $\mathbf{x}$

$\nabla_{\mathbf{x}} f(\mathbf{x})$ points in the direction where f **increases most rapidly**.
So $-\nabla_{\mathbf{x}} f(\mathbf{x})$ is the direction of steepest descent

Gradient descent algorithm

Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η

while true **do**

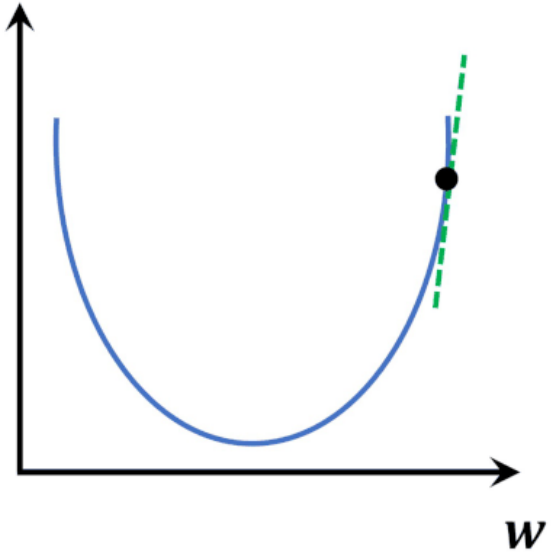
  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$

  if converge **then**

    return $\mathbf{x}_{k+1}$

  end

end



Animation sources: https://datahacker.rs/003-pytorch-how-to-implement-linear-regression-in-pytorch/

Gradient descent algorithm

Key points:

- $-\eta \nabla_x f(\mathbf{x}_k)$: The product of the *learning rate* and the *gradient*, which determines the ***direction*** and ***step size*** to move towards minimizing the cost.

- Common stopping criteria:
  - Maximum number of iterations is reached.
  - The percentage or absolute change in f is below a threshold.
  - The percentage or absolute change in the variable $\mathbf{x}$ is below a threshold.

Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η

while true **do**

  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$

  if converge **then**

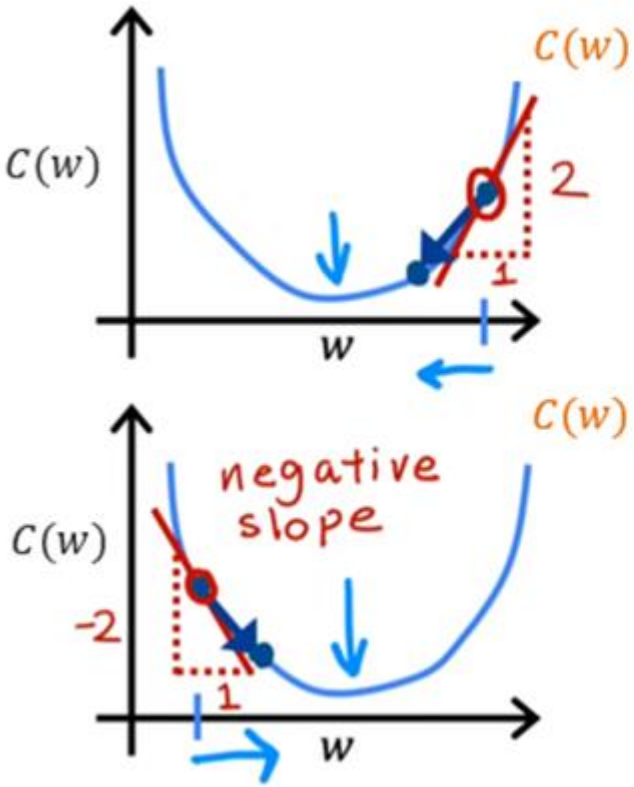
    return $\mathbf{x}_{k+1}$

  end

end

Gradient descent algorithm

Gradient Descent:
- Initialize $\mathbf{x}_0$ and learning rate η
- **while** true **do**
  - Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$
  - **if** converge **then**
    - **return** $\mathbf{x}_{k+1}$
  - **end**
- **end**



$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

$$w = w - \eta \cdot (positive\ number)$$

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

$$w = w - \eta \cdot (negative\ number)$$

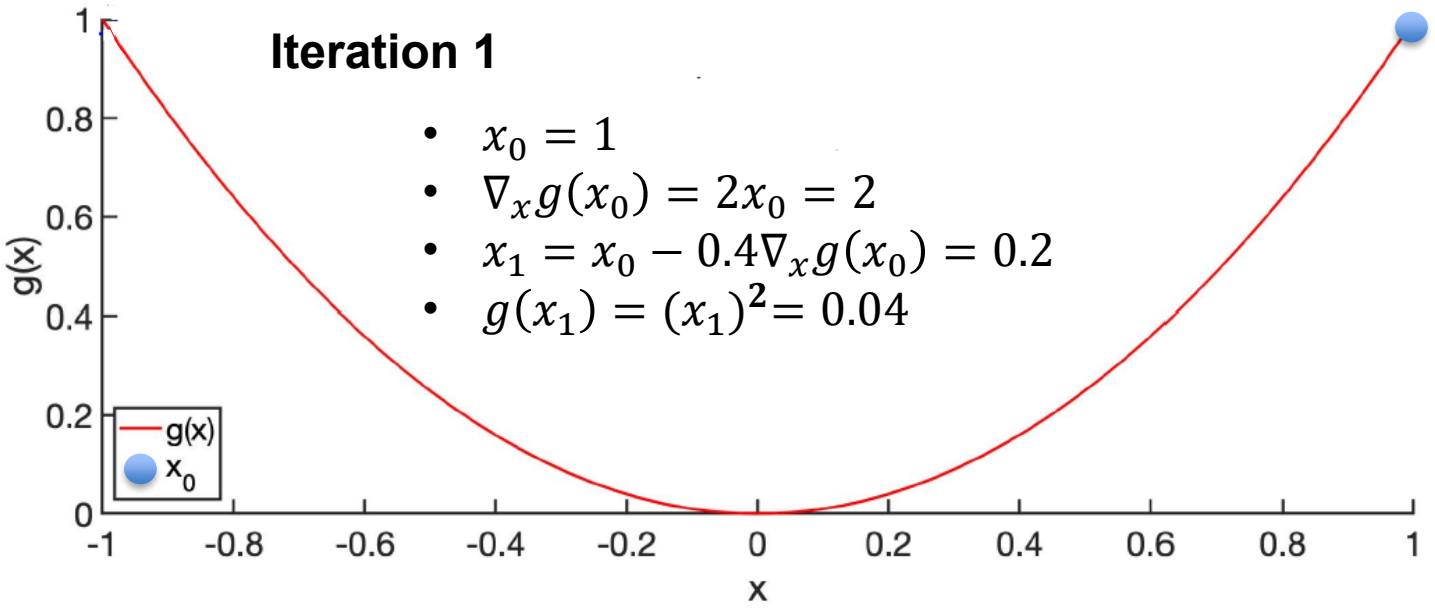
Modified from https://www.youtube.com/watch?v=PKm61nrqpCA by Dr Andrew Ng

Gradient descent (example)

Qn: Find x to minimize $g(x) = x^2$.
Initialize $x_0 = 1$, learning rate $\eta = 0.4$

1. Compute the gradient: $\nabla_x g(x) = 2x$
2. At each iteration, $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$

Gradient Descent:
Initialize $\mathbf{x}_0$ and learning rate η
while true **do**
  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$
  if converge **then**
    return $\mathbf{x}_{k+1}$
  end
end

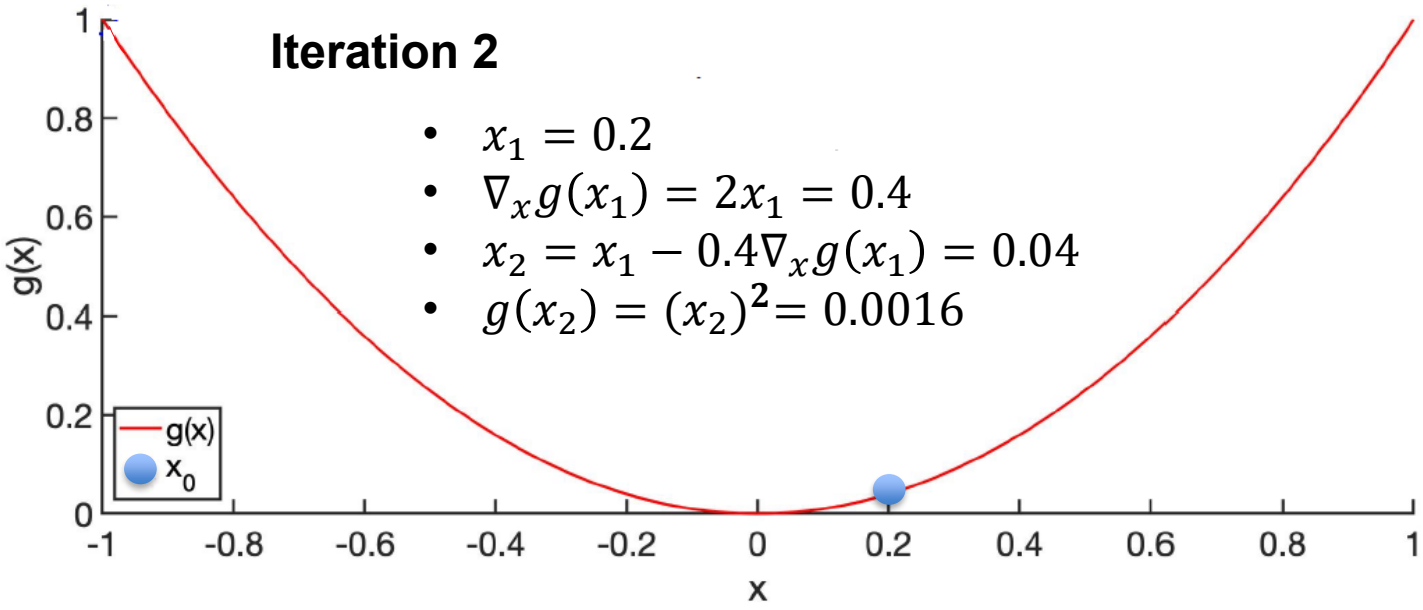


Gradient descent (example)

Qn: Find x to minimize $g(x) = x^2$.
Initialize $x_0 = 1$, learning rate $\eta = 0.4$

1. Compute the gradient: $\nabla_x g(x) = 2x$
2. At each iteration, $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$

Gradient Descent:
Initialize $\mathbf{x}_0$ and learning rate η
while true **do**
  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$
  if converge **then**
    return $\mathbf{x}_{k+1}$
  end
end

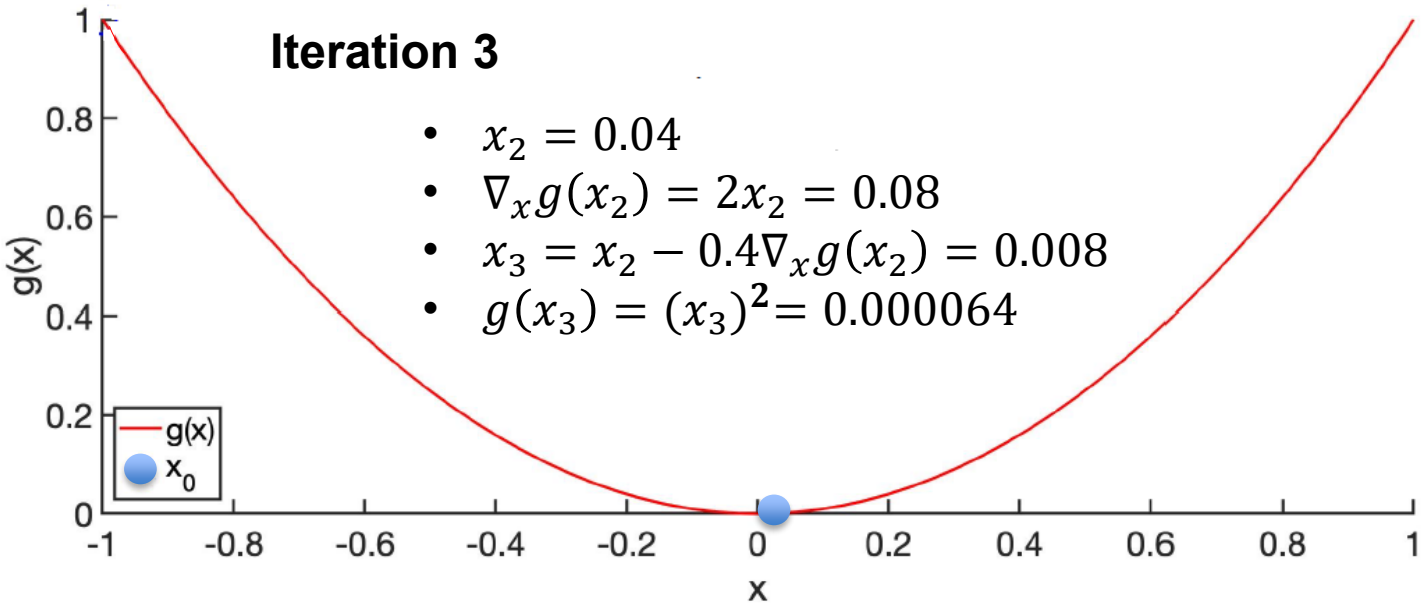


Gradient descent (example)

Qn: Find x to minimize $g(x) = x^2$.
Initialize $x_0 = 1$, learning rate $\eta = 0.4$

1. Compute the gradient: $\nabla_x g(x) = 2x$
2. At each iteration, $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$

Gradient Descent:
Initialize $\mathbf{x}_0$ and learning rate η
while true **do**
  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$
  if converge **then**
    return $\mathbf{x}_{k+1}$
  end
end

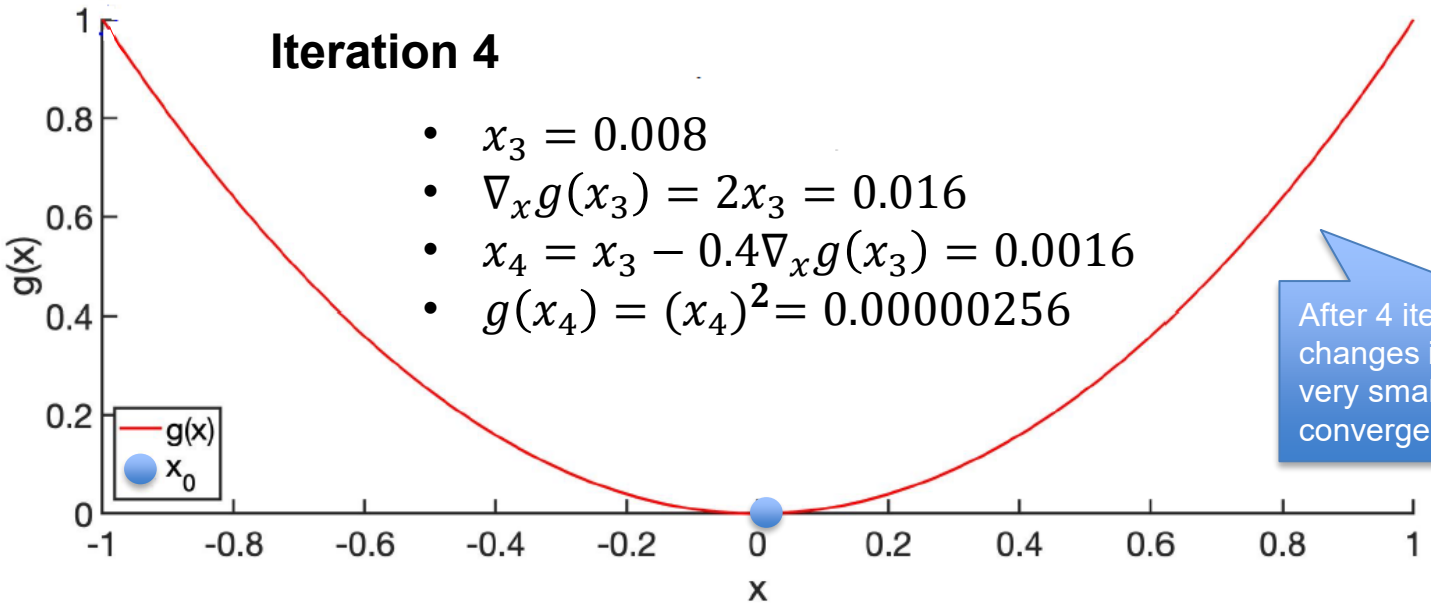


Gradient descent (example)

Qn: Find x to minimize $g(x) = x^2$.
Initialize $x_0 = 1$, learning rate $\eta = 0.4$

1. Compute the gradient: $\nabla_x g(x) = 2x$
2. At each iteration, $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$

Gradient Descent:
Initialize $\mathbf{x}_0$ and learning rate η
while true **do**
  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$
  if converge **then**
    return $\mathbf{x}_{k+1}$
  end
end



Reference (similar animation): https://www.youtube.com/watch?v=huyBoeTkm8I

Gradient descent algorithm overview

Gradient descent takes **larger steps** when the slope is **steep**, allowing for faster progress in the initial phases of optimization.

As the slope **flattens**, gradient descent takes **smaller steps**, enabling precise convergence towards the minimum ("Goal") without overshooting.

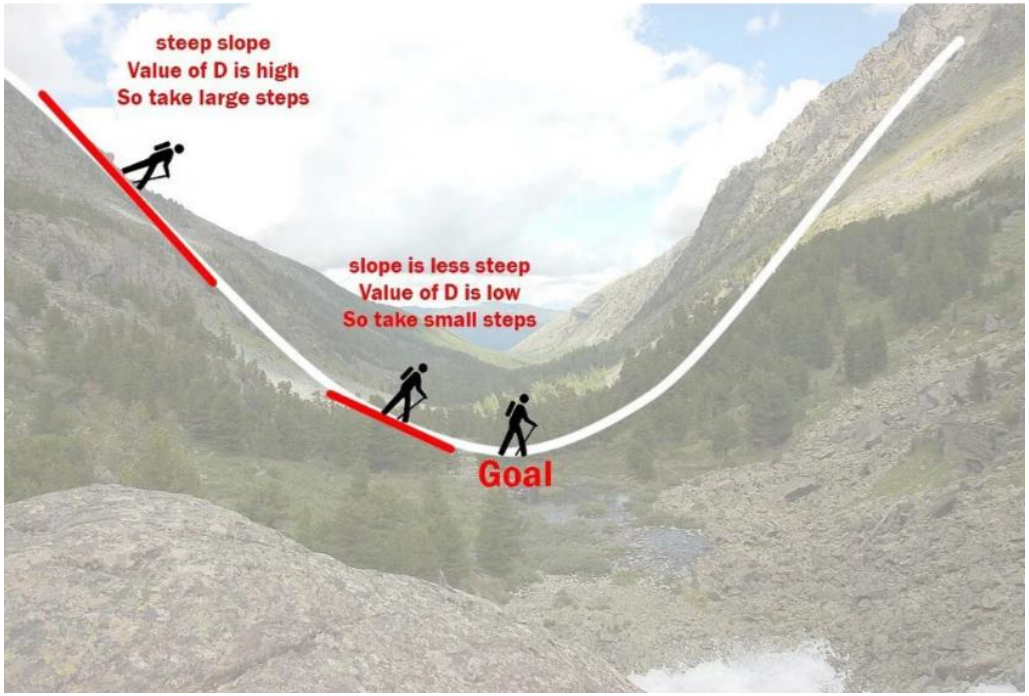


Image source: https://www.inspiritai.com/

Class practice 1: Gradient descent

Qn: Minimize $f(x) = (x_1 - 1)^2 + (x_2 - 2)^2$

with respect to $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

Initialize: $x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Learning rate: $\eta = 0.1$

Gradient $\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$

Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η

while true **do**

  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$

  if converge **then**

    return $\mathbf{x}_{k+1}$

  end

end

Class practice 2: Gradient descent

Qn: Suppose we are minimizing $g(x) = x^2$. We initialize x to be 1. Using gradient descent with learning rate $\eta = 1$, perform the first few steps of the algorithm and observe what happens.

Does the value of x get closer to the minimum, or does it keep bouncing back and forth? Why?

Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η
while true **do**
  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$
  if converge **then**
    return $\mathbf{x}_{k+1}$
  end
end

Gradient descent algorithm

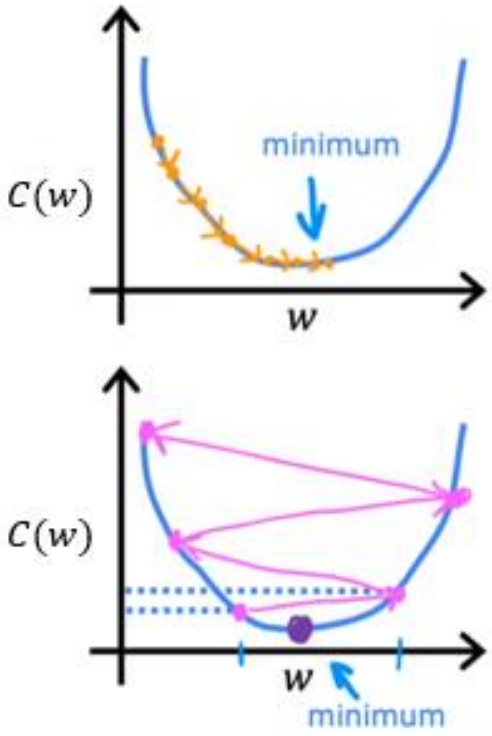
Choose suitable learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

If η is too small...
Gradient descent may be slow.

If η is too large...

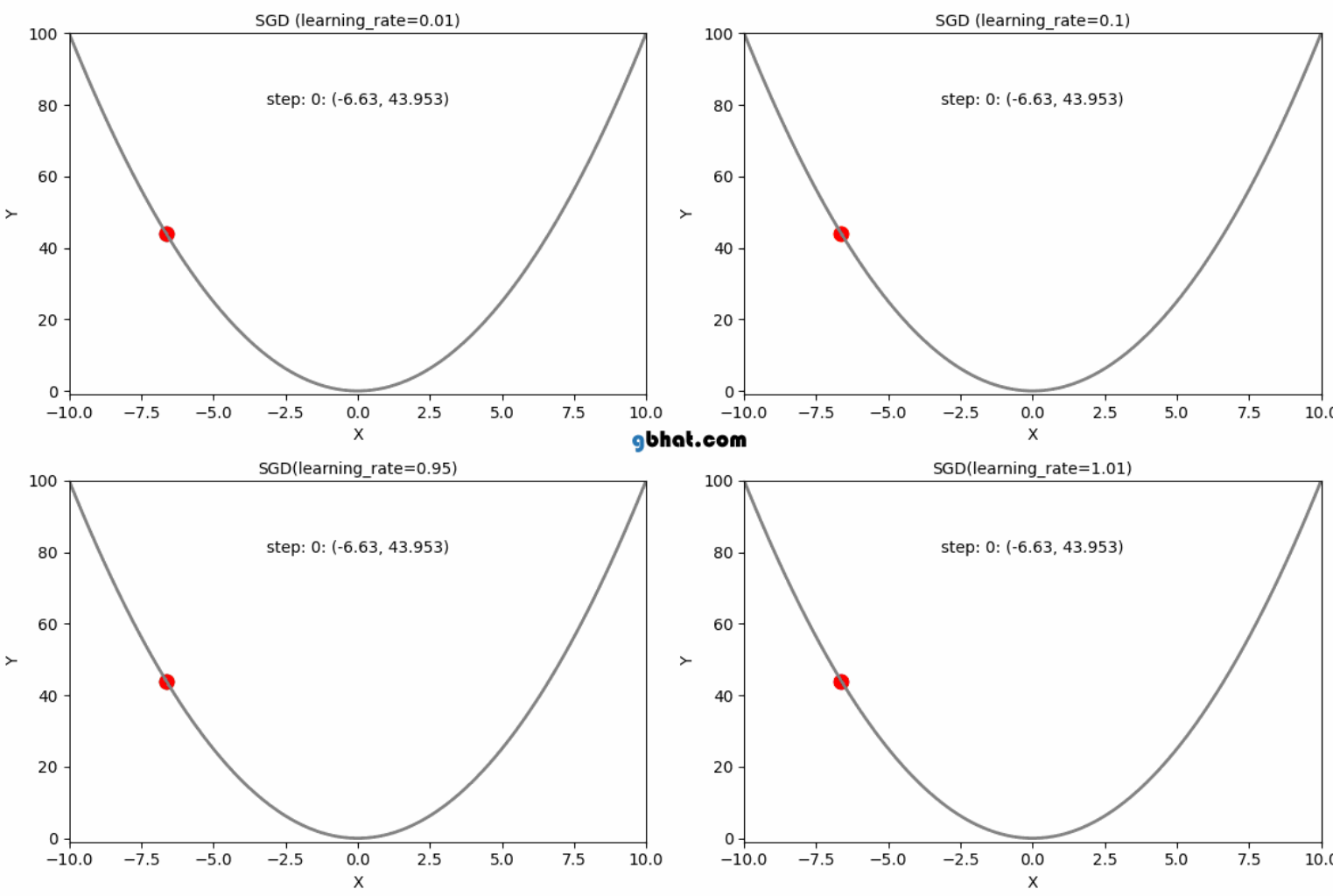
Gradient descent may:
- Overshoot, never reach minimum
- Fail to converge, diverge



Source: https://www.youtube.com/watch?v=k0h8emRAAHE

Gradient descent algorithm

Different learning rates comparison



Source: https://gbhat.com/machine_learning/gradient_descent_learning_rates.html

Convex optimization: How to solve

- **Unconstrained** and **differentiable** convex functions:
  Gradient descent converges to the **global minimum** with a sufficiently small step size $(\text{i.e., learning rate } \eta)$.

- **Constrained** and **differentiable** convex functions:
  Projected gradient descent is used when the domain is constrained to a convex set.

- **Non-differentiable** convex functions (*not covered in this course*):
  ϵ-subgradient methods can be applied

* In non-convex functions, gradient descent might get stuck in local minima but sometimes finds good solutions in practice.

Projected gradient descent

Projected Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η

while true **do**

  Compute:

    $\mathbf{z}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$

    $\mathbf{x}_{k+1} \leftarrow \Pi_C(\mathbf{z}_{k+1})$ // *Project* $\mathbf{z}_{k+1}$ *onto constraint set*

  if converge **then**

    return $\mathbf{x}_{k+1}$

  end

end

$\Pi_C(\mathrm{z})$: the projection of a point z onto a convex set C

e.g., $\Pi_C(\mathbf{z}) = \underset{\mathbf{x} \in C}{\operatorname{argmin}} \|\mathbf{x} - \mathbf{z}\|_2$

Take the point z (which might be outside the constraint set C), and find the *closest point* in C to it — using *Euclidean distance* (L2 norm), denoted by $\|\bullet\|$.

Projected gradient descent: Example

Qn: Minimize $g(x) = (x-2)^2$.
Subject to: $x \geq 0$ (feasible region $C = [0, +\infty)$)
Initialize $x_0 = -1$, learning rate $\eta = 0.15$

1. Compute the gradient: $\nabla_x g(x) = 2x - 4$
2. At each iteration: $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$ // gradient step (tentative)
   $\mathbf{x}_{k+1} \leftarrow \Pi_C(\mathbf{z}_{k+1})$ // projection step

Iteration 1

Gradient step (tentative):

- $x_0 = -1$
- $\nabla_x g(x_0) = 2x_0 - 4 = -6$
- $z_1 = x_0 - 0.15\nabla_x g(x_0) = -0.1$

Projection step:

Since $z_1 = -0.1 < 0$, we project it to its closet point in feasible region C
$x_1 \leftarrow \Pi_C(-0.1) = 0$

Projected gradient descent: Example

Qn: Minimize $g(x) = (x-2)^2$.
Subject to: $x \geq 0$ (feasible region $C = [0, +\infty)$)
Initialize $x_0 = -1$, learning rate $\eta = 0.15$

1. Compute the gradient: $\nabla_x g(x) = 2x - 4$
2. At each iteration: $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$ // gradient step (tentative)
$\mathbf{x}_{k+1} \leftarrow \Pi_C(\mathbf{z}_{k+1})$ // projection step

Iteration 2

Gradient step (tentative):

- $x_1 = 0$
- $\nabla_x g(x_1) = 2x_1 - 4 = -4$
- $z_2 = x_1 - 0.15\nabla_x g(x_1) = 0.6$

Projection step:

Since $z_2 = 0.6 \geq 0$, its projection is itself
$x_2 \leftarrow \Pi_C(0.6) = 0.6$

Projected gradient descent: Example

Qn: Minimize $g(x) = (x-2)^2$.
Subject to: $x \geq 0$ (feasible region $C = [0, +\infty)$)
Initialize $x_0 = -1$, learning rate $\eta = 0.15$

1. Compute the gradient: $\nabla_x g(x) = 2x - 4$
2. At each iteration: $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$ // gradient step (tentative)
   $\mathbf{x}_{k+1} \leftarrow \Pi_C(\mathbf{z}_{k+1})$ // projection step

Iteration 3

Gradient step (tentative):

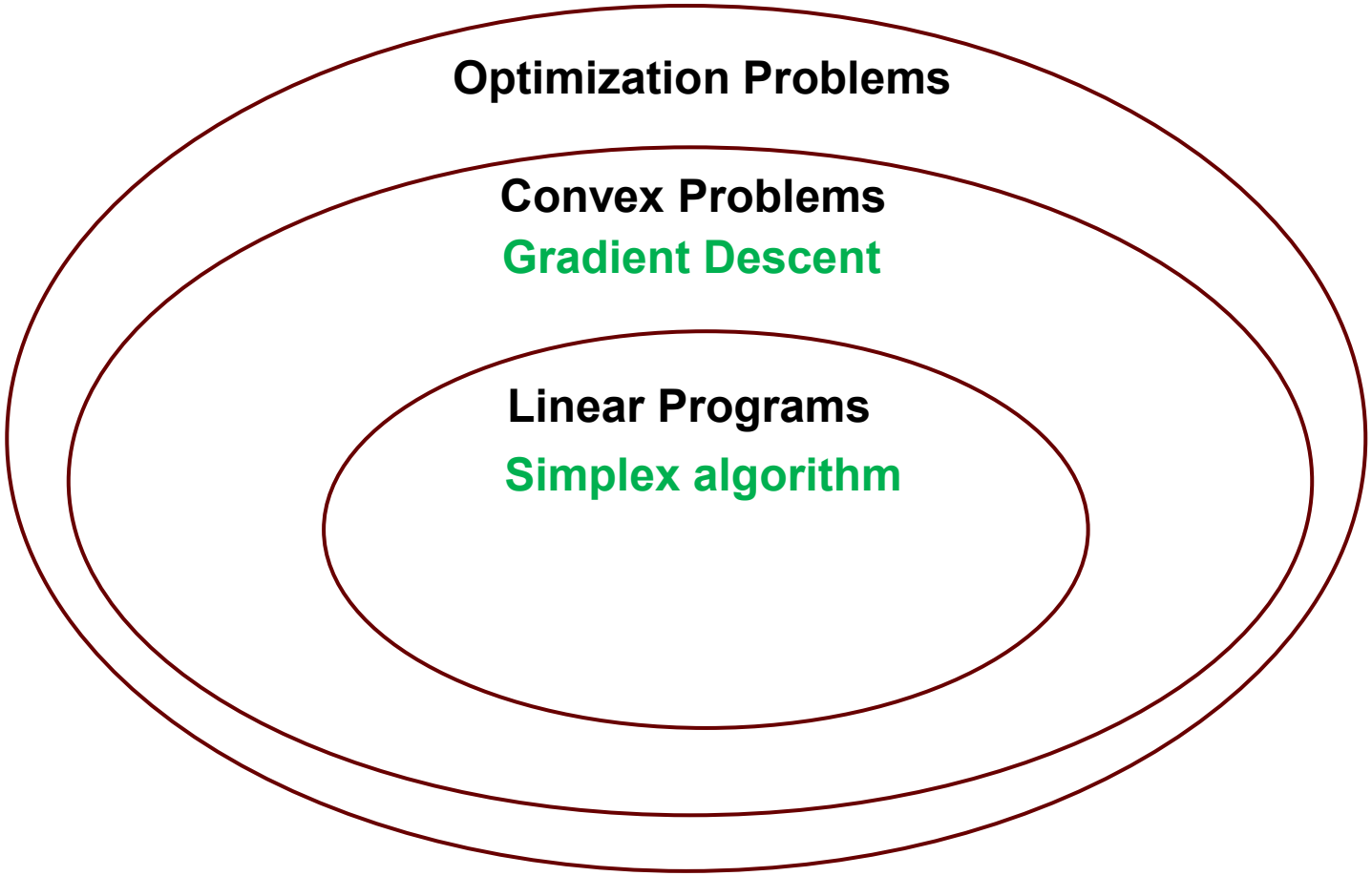
- $x_2 = 0.6$
- $\nabla_x g(x_2) = 2x_2 - 4 = -2.8$
- $z_3 = x_2 - 0.15\nabla_x g(x_2) = 1.02$

Projection step:

Since $z_3 = 1.02 \geq 0$, its projection is itself
$x_3 \leftarrow \Pi_C(1.02) = 1.02$

After 3 iterations, we are at 1.02, continuously approaching the optimal solution at x=2.

Summary



References & further readings

Introduction of gradient descent by Andrew Ng:
https://www.youtube.com/watch?v=PKm61nrqpCA

Classic book of convex optimization by Boyd and Vandenberghe:
https://web.stanford.edu/~boyd/cvxbook/

Stanford University course: Convex optimization, taught by Stephen Boyd.
https://www.youtube.com/watch?v=McLq1hEq3UY